ЦИКТ ЦЕНТР ИНФОРМАЦИОННО-КОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ ГАПОУ КВ № 11

«КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА №11»

# Разработка мобильных приложений

# Что такое СКВ

**Система управления версиями** (от англ. **Version Control System, VCS** или **Revision Control System**) — программное обеспечение для облегчения работы с изменяющейся информацией

ЦИКТ ЦЕНТР ИНФОРМАЦИОННО-КОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ ГАПОУ КП № 11

# Что такое СКВ

«КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА №11»

Система управления версиями позволяет хранить несколько версий одного и того же проекта, при необходимости возвращаться к более ранним версиям, определять, кто и когда сделал то или иное изменение, и многое другое

# СКВ

Система контроля версий фиксирует изменения файлов во времени: можно сравнивать версии, видеть автора изменения и возвращаться к устойчивому состоянию.



# Что такое Git

Распределённая система управления версиями.

Проект был создан Линусом Торвальдсом для управления разработкой ядра **Linux**, первая версия выпущена 7 апреля 2005 года

**Git** – распределенная система контроля версий (DVCS)



# Репозиторий

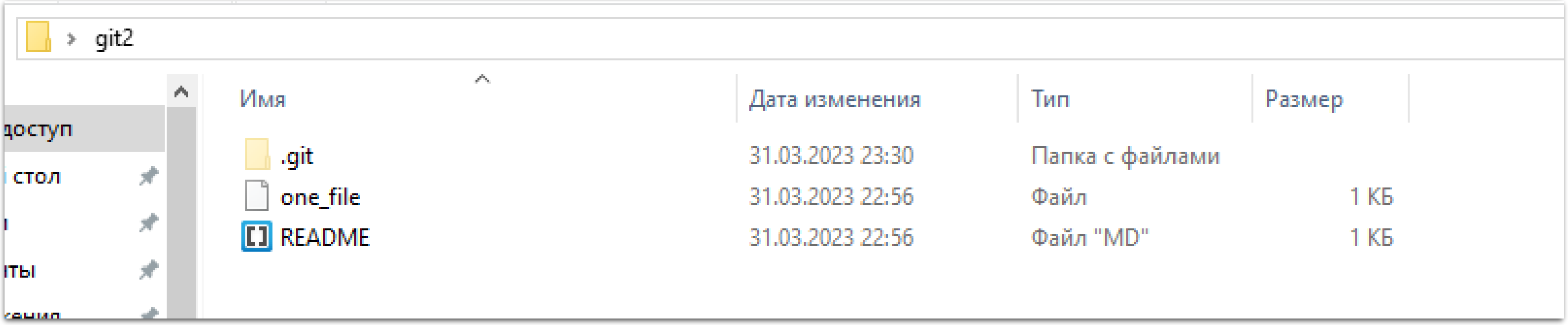
**Git** хранит информацию в структуре данных – репозиторий (**Repository**)

Репозиторий хранит:

- **Commit objects –** Содержит набор файлов отображающий состояние проекта в текущую точку времени
- Набор ссылок на **commit objects** называемых **heads**

# Репозиторий

Репозиторий хранится в той же директории, что и сам проект в поддиректории **.git**



# Репозиторий

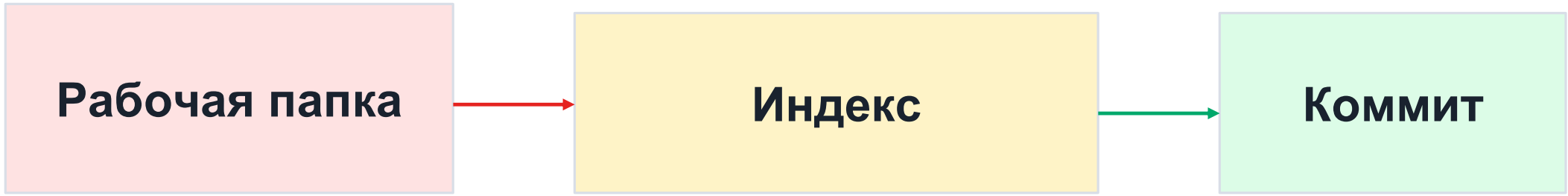
## Основной цикл

```
git init

git add .

git commit -m "start"

git push -u origin main
```



# Репозиторий

## Зачем Git нужен разработчику

### 1 Проблема

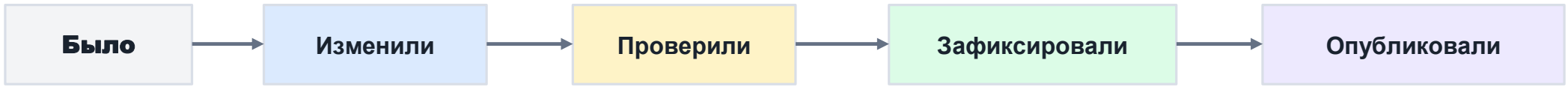
Проект меняется: код, изображения, настройки и документация быстро расходятся между версиями.

### 2 Решение

Система контроля версий хранит историю изменений и позволяет вернуться к нужному состоянию.

### 3 Командная работа

Разные студенты могут делать задачи параллельно, а затем объединять изменения.



# Репозиторий Git

**Head –** ссылка на **commit object.**

Каждый **head** имеет имя

В каждом репозитории существует **head** называемый **- master**

Репозиторий может содержать любое количество **head**

Выбранный **head** называют **– current head**

# Программное обеспечение

Для работы с **Git** используют:

**Linux, OS X – Terminal**

**Windows 10 – PowerShell**




Сторонние клиенты:

**Git Scm** - https://git-scm.com/

- **Git bash**
- **Git GUI**

**GitHub** – для работы с ресурсом **GitHub** https://github.com/

# Установка и первичная настройка

### 1 Установка

Windows: Git for Windows. Linux/macOS: пакетный менеджер или официальный установщик.

### 2 Проверка

Команда git --version показывает установленную версию Git.

### 3 Идентификация

Имя и e-mail попадут в историю коммитов. Для учебного курса используйте реальные ФИО/учебную почту.

**Команды**

```
git --version
git config --global user.name "Иван Иванов"
git config --global user.email ivanov@example.ru
git config --global init.defaultBranch main
git config --list --show-origin
```

# Установка и первичная настройка

После установки git, необходимо его настроить.

Откройте терминал и запустите команды:

**git config --global user.name "My Name"**

**git config --global user.email myEmail@example.com**

# Создать или склонировать репозиторий

## A Создать новый

Используется для своего проекта: сначала папка, затем git init.

Новый проект

```
mkdir my-app
cd my-app
git init
git branch -M main
```

## B Скачать существующий

Используется для учебного шаблона, командного проекта или чужого репозитория.

Готовый проект

```
git clone https://github.com/user/project.git
cd project
git status
```

**Важно: в примерах используйте адрес своего репозитория, а не чужой IP или учебный шаблон.**

# Инициализация проекта

**Git** хранит свои файлы и историю прямо в папке проекта.

Чтобы создать новый репозиторий, нужно открыть терминал, зайти в папку нашего проекта и выполнить команду **init.** Это включит приложение в этой конкретной папке и создаст скрытую директорию **.git,** где будет храниться история репозитория и настройки

Пример: **cd Desktop/MyProject/**

**git init**

Терминал должен вернуть сообщение об успешном выполнении: **Initialized empty Git repository in /home/user/Desktop/git_exercise/.git/**

# Определение статуса

**Status** — это команда, которая показывает информацию о текущем состоянии репозитория: актуальна ли информация на нём, нет ли чего-то нового, что поменялось, и так далее

Запуск **git status** на нашем свежесозданном репозитории должен выдать:

**On branch master Initial commit Untracked files: (use "git add ..." to include in what will be committed) MyProject.txt**

*Сообщение говорит о том, что файл **hello.txt** неотслеживаемый. Это значит, что файл новый и система еще не знает, нужно ли следить за изменениями в файле или его можно просто игнорировать. Для того, чтобы начать отслеживать новый файл, нужно его специальным образом объявить*

# git status и git diff

**status  git status**

показывает состояние рабочей папки и индекса: новые, изменённые, подготовленные файлы.

**diff  git diff**

показывает, какие строки изменились. Это проверка перед добавлением в индекс и коммитом.

**Команды проверки**

```
git status
git diff
git diff --staged
```

```
+ новая строка
- удалённая строка
@@ участок файла @@
```

# Подготовка файлов

В **git** есть концепция области подготовленных файлов. Можно представить ее как холст, на который наносят изменения, которые нужны в коммите. Сперва он пустой, но затем мы добавляем на него файлы (или части файлов, или даже одиночные строчки) командой **add** и, наконец, коммитим все нужное в репозиторий (создаем слепок нужного нам состояния) командой **commit**.

В нашем случае у нас только один файл, так что добавим его:

**git add MyProject.txt**

Для добавления всех файлов **git add -A** или **git add .**

# Фиксация изменений

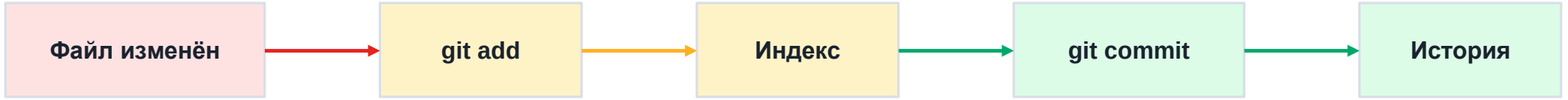
Чтобы зафиксировать изменения, нужно хотя бы одно изменение в области подготовки (только что создали его при помощи **git add**), после которого можно коммитить:

**git commit -m ”My first commit”**

Эта команда создаст новый коммит со всеми изменениями из области подготовки (добавление файла **MyProject.txt**). Ключ **-m** и сообщение **“My first commit”** — это созданное пользователем описание всех изменений, включенных в коммит

*Считается хорошей практикой делать коммиты часто и всегда писать содержательные комментарии

# git add и git commit




Команды

```
git add README.md
git add .
git commit -m "Добавлен README"
git log --oneline --decorate
```

✓ **Хороший коммит**

одна законченная мысль: “исправлена валидация e-mail”, а не “изменения”.

# Сообщения коммитов

Сообщение должно помогать понять “что изменено” и “зачем”.

Для учебных работ достаточно короткой формы.

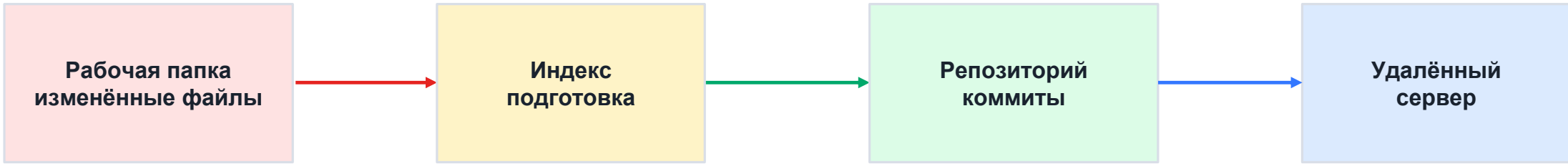
| × Плохо | ✓ Хорошо |
| --- | --- |
| update | Добавлен экран авторизации |
| fix | Исправлена ошибка в расчёте суммы |
| работа | Обновлён .gitignore для Android |
| новое | |

**Пример**

```
git commit -m "Добавлена проверка пустого логина"
```

# Три состояния файла в Git

Начинающему проще понять Git через три области: рабочая папка, индекс и репозиторий.



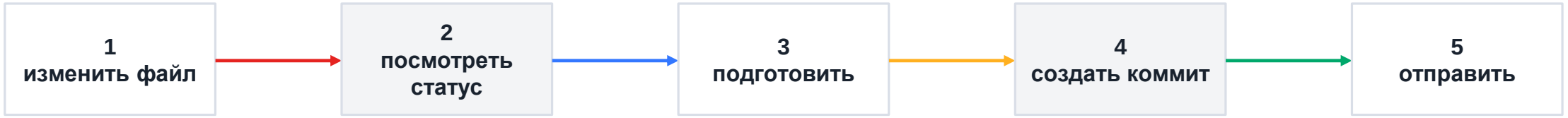

Как это выглядит в терминале

```
git status
git add file.txt
git commit -m "описание"
git push -u origin main
```

**? Индекс нужен**

чтобы выбрать только нужные изменения для следующего коммита, а не фиксировать всё подряд.

# Базовый сценарий работы




Шпаргалка для практики

```
# проверить
git status
# подготовить
git add .
# зафиксировать
git commit -m "Добавлен экран входа"
# отправить
git push
```

# Сетевой (удаленный) репозиторий

Чтобы загрузить что-нибудь в удаленный репозиторий, сначала нужно к нему подключиться. Чтобы связать наш локальный репозиторий с репозиторием на **Git**, выполним следующую команду в терминале. Обратите внимание, что нужно обязательно изменить **URI** репозитория на свой

**git remote add origin** https://5.63.153.197/mxtube/Test.git

Вместо **origin**, можно указать свое имя для подключения

**git remote –v** просмотр созданных подключений

# Сетевой репозиторий: origin и push

origin — обычное имя удалённого репозитория. Оно не обязательно, но используется как стандарт в учебных примерах.

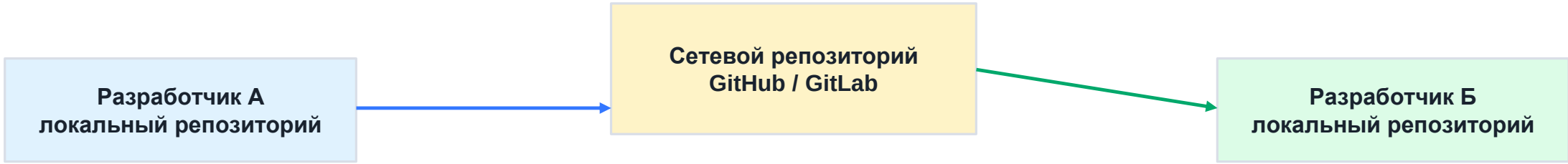



## Подключение и отправка

```
git remote add origin https://github.com/user/project.git
git remote -v
git push -u origin main
```

# Локальная, централизованная и распределённая модель

В Git каждый разработчик имеет локальную копию истории проекта. Большинство операций выполняются без сети; сеть нужна для обмена изменениями.




**↑ push**

отправляет локальные коммиты в удалённый репозиторий.

**↓ fetch / pull**

получает изменения из удалённого репозитория.

**↔ merge**

объединяет истории, если ветки разошлись.

# Локальная, централизованная и распределённая модель



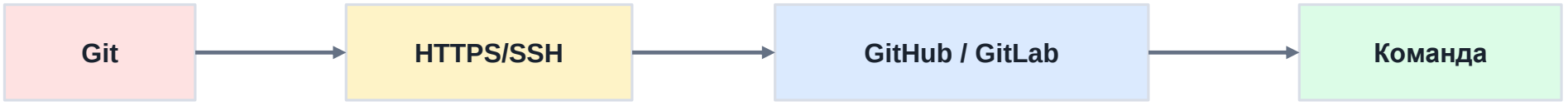
## Git и GitHub: не одно и то же

**Git  Git**

программа и формат репозитория: локальные коммиты, ветки, история, обмен изменениями.

**Web  GitHub / GitLab / Gitea**

веб-сервисы для хранения удалённых репозиториев, pull/merge request, issues и совместной работы.



# .gitignore: что не нужно отправлять

В репозиторий отправляют исходный код и настройки проекта, но не временные файлы, кэш, сборки и секреты.

Пример .gitignore

```
# Android / Gradle
.gradle/
build/
local.properties
*.apk
*.aab

# IDE
.idea/
*.iml

# секреты
.env
keys.properties
```

**! Почему это современно**

Git хранит историю. Если пароль, токен или ключ попал в коммит, его трудно “стереть” полностью.

**✓ Правило для студентов**

перед add всегда выполняем git status и убеждаемся, что в списке нет лишних файлов.

# Отправка изменений

Команда, предназначенная для отправки изменений - **push**. Она принимает два параметра: имя удаленного репозитория и ветку, в которую необходимо внести изменения (**master** — это ветка по умолчанию для всех репозиториев).

# Клонирование репозитория

Для скачивания своих или проектов других пользователей используют команду **clone**. Перед скачиванием проекта необходимо перейти в директорию куда будет скачен проект, с помощью команды **cd**

```
git clone https://5.63.153.197/mxtube/Test.git
```

# Запрос изменений

Если вы сделали изменения в вашем репозитории, другие пользователи могут скачать изменения при помощи команды **pull.**

**git pull origin master**

# Ветки (Branch)

Во время разработки новой функциональности считается хорошей практикой работать с копией оригинального проекта, которую называют веткой. Ветви имеют свою собственную историю и изолированные друг от друга изменения до тех пор, пока вы не решаете слить изменения вместе. Это происходит по набору причин:

- Уже рабочая, стабильная версия кода сохраняется.
- Различные новые функции могут разрабатываться параллельно разными программистами.
- Разработчики могут работать с собственными ветками без риска, что кодовая база поменяется из-за чужих изменений.
- В случае сомнений, различные реализации одной и той же идеи могут быть разработаны в разных ветках и затем сравниваться.

# Ветки (Branch)

Во время разработки новой функциональности считается хорошей практикой работать с копией оригинального проекта, которую называют веткой. Ветви имеют свою собственную историю и изолированные друг от друга изменения до тех пор, пока вы не решаете слить изменения вместе. Это происходит по набору причин:

- Уже рабочая, стабильная версия кода сохраняется.
- Различные новые функции могут разрабатываться параллельно разными программистами.
- Разработчики могут работать с собственными ветками без риска, что кодовая база поменяется из-за чужих изменений.
- В случае сомнений, различные реализации одной и той же идеи могут быть разработаны в разных ветках и затем сравниваться.

# Работа с ветками

1. Создание ветки:

Основная ветка в каждом репозитории называется **master**.

Чтобы создать еще одну ветку, используем команду **branch <name>**

**git branch Session2**

Для просмотра существующих веток используют команду **git branch**

2. Переключение между ветками:

**git checkout имя_ветки**

3. Объединение веток:

**git merge имя_ветки**

4. **git branch -d** имя_ветки

# fetch, pull и синхронизация

### 1 git fetch

скачивает сведения об изменениях, но не объединяет их с вашей веткой.

### 2 git pull

получает изменения и сразу объединяет их с текущей веткой.

### 3 git push

отправляет ваши локальные коммиты на сервер.

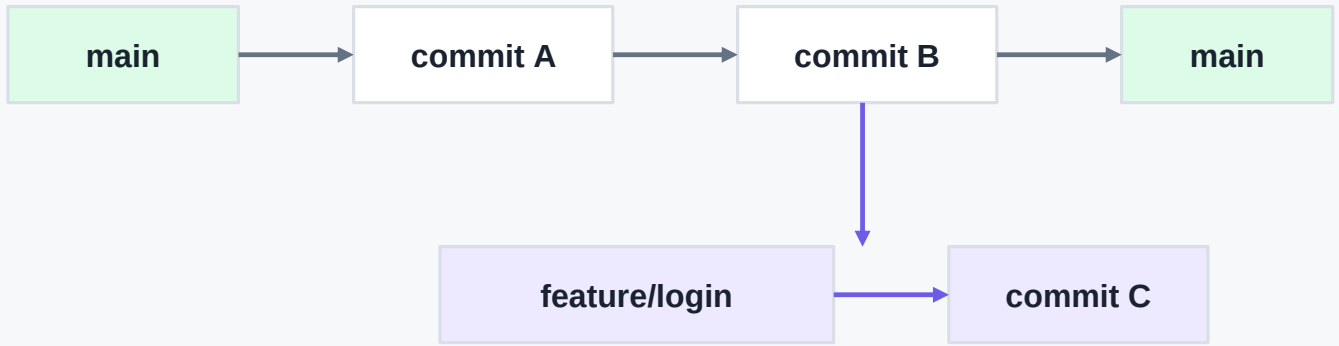
**Команды обмена**

```
git fetch origin
git pull origin main
git push origin main
```

**Перед push в командном проекте полезно выполнить pull, чтобы не отправлять изменения поверх устаревшей истории.**

# Ветки: современная практика

Ветка — это удобная “линия работы”. В ней можно делать задачу, не ломая основную стабильную версию.




**! Важное обновление**

В старых материалах часто используется master и checkout. В современных учебных примерах лучше показывать main и git switch.

# Работа с ветками

**Современный сценарий**

```
git branch
git switch -c feature/login
# работа с файлами
git add .
git commit -m "Добавлен экран входа"
git switch main
git merge feature/login
git branch -d feature/login
```

### → Что происходит

switch -c создаёт ветку и сразу переключает на неё. merge переносит результат работы в выбранную ветку.

### ✓ Когда использовать ветку

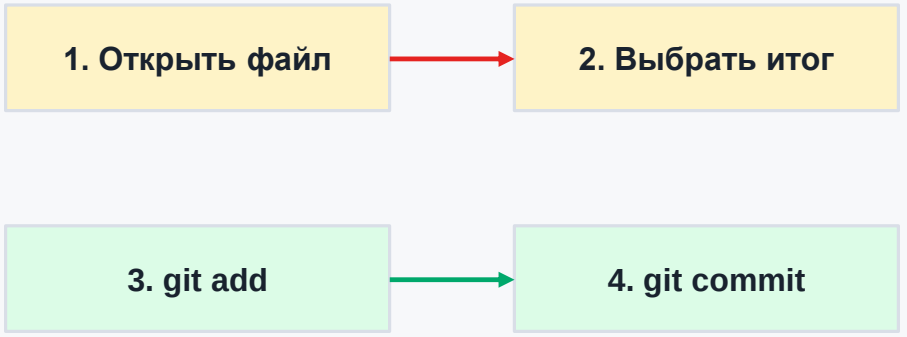
каждая новая функциональность, исправление ошибки, эксперимент или отдельная лабораторная работа.

# Конфликт слияния

Конфликт возникает, когда две ветки изменили один и тот же участок файла по-разному. Git показывает маркеры, а человек выбирает итог.

Файл с конфликтом

```
<<<<<<< HEAD
текущий вариант
=======
вариант из другой ветки
>>>>>>> feature/login
```




После исправления

```
git status
# исправить файл вручную
git add App.kt
git commit
```

# Отмена изменений: аккуратно

### 1 restore

безопасно отменяет изменения в рабочей папке или убирает файл из индекса.

### 2 revert

создаёт новый коммит, который отменяет выбранный старый коммит. Подходит для общей истории.

### 3 reset

двигает указатель ветки. Может потерять изменения; использовать осторожно и только понимая последствия.

**Команды**

```
git restore file.txt
git restore --staged file.txt
git revert <hash>
# осторожно:
git reset --hard <hash>
```

# Ссылки на ресурсы

https://github.com/

http://gitlab.com/

https://git-scm.com/

https://bitbucket.org/

https://gogs.io/

и другие















- https://habr.com/ru/articles/552872/